CS402/MA492 Cryptography

Tobias Rossmann
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

Lecture 6:

Cryptography in Python, part 1

What this isn't

- This is not an introduction to computer science, computer programming, or Python.
- This won't teach you how to write production-level cryptographic code.

What this is

- A quick review of Python basics ("CS103 for the seriously impatient").
- A tutorial on how to use the Python module cs402 (http://www.maths.nuigalway.ie/~rossmann/cs402/cs402.py).
- Some ideas that will help you get started working on the first assignment. (*Note:* Assignment 1 will be released on Monday, 3 February 2020.)

Python

- The Python programming language (https://www.python.org/) has been continuously developed since the 1980s.
  Most recent major version: Python 3.0 (2008).
- Python is very easy to use and based on an intuitive mathematical syntax.

- Python natively supports arbitrary-precision arithmetic needed for cryptography and computational number theory.
- For serious number crunching, the performance of Python code cannot usually compete with low-level implementations (e.g. in C).

Installing and using Python

- Python is available (at least) in the PC suite in Áras de Brún (ground floor).
- The recommended way of using Python on your own computer is via the Anaconda (https://www.anaconda.com/) distribution.
- Python can be used from the command line or using a graphical notebook interface (Project Jupyter (https://jupyter.org/)).
- This presentation *is* a Jupyter notebook.

Jupyter notebooks

- A notebook consists of a sequence of **cells**.
- A cell either contains **text** or (Python) **code**.
- A text cell contains markdown (https://daringfireball.net/projects/markdown/) formatted text.
- A code cell can be executed ("run") using the notebook's Python kernel.
- Running text cells 'renders' the text.

- At each time, the notebook is either in **edit mode** or in **command mode**.
- In command mode, you can move around the notebook and operate on cells (e.g. copy and paste, split or merge cells, etc.).
- In edit mode, the content of the current cell can be modified.

Python as a calculator

We can use the Python interpreter as a powerful calculator, using

- `+`, `-`, `*`, `/` for the usual arithmetic operations,
- `//` for integer (= floor) division
- `%` for the modulo operation, and
- `**` for exponentiation.

What is the last decimal digit of 2^{200}?

In [1]:

```
(2**200) % 10
```

Out[1]:

6

In [2]:

```
2**200
```

Out[2]:

```
1606938044258990275541962092341162602522202993782792835301376
```

- Ordinary division (`/`) usually results in **floating point numbers**.
- These have limited precision. In contrast, `//` integer division always produces integer objects.
- Python integers are only restricted by the available memory.

In [3]:

```
3/2
```

Out[3]:

```
1.5
```

In [4]:

```
3//2
```

Out[4]:

```
1
```

Values (e.g. results of computations) can be assigned to **variables** for later use.

In [5]:

```
a = 2**(3**4)
print(a % 3)
```

```
2
```

Conditionals

Values can be compared using `==` ("is equal to"), `!=` ("is not equal to"), `<` , `<=` , etc.

In [6]:

```
2**5 > 5**2
```

Out[6]:

```
True
```

In [7]:

```
4**5 > 4**5
```

Out[7]:

```
False
```

`if` statements can be used for conditional program flow.

In [8]:

```
x = int(input('Enter a number: '))
if x < 0:
    print('negative')
```

```
Enter a number: -5
negative
```

In [9]:

```
x = int(input('Enter a number: '))
if x < 0:
    print('negative')
else:
    print('non-negative')
```

```
Enter a number: 5
non-negative
```

In [10]:

```
x = int(input('Enter a number: '))
if x < 0:
    print('negative')
elif x > 0:
    print('positive')
else:
    print('zero')
```

```
Enter a number: 0
zero
```

Important. In Python, related pieces of code (e.g. clauses of an `if` statement) are grouped together based on their indentation level.

Lists

Lists are ordered collections of objects.

In [11]:

```
L = [42, 7, 0, 7]
```

In [12]:

```
L[0]
```

Out[12]:

```
42
```

In [13]:

```
L[1] = -4
L
```

Out[13]:

```
[42, -4, 0, 7]
```

In [14]:

```
len(L)
```

Out[14]:

```
4
```

In [15]:

```
L.append(12)
L
```

Out[15]:

```
[42, -4, 0, 7, 12]
```

In [16]:

```
L.extend([9,2,9,7,11])
L
```

Out[16]:

```
[42, -4, 0, 7, 12, 9, 2, 9, 7, 11]
```

In [17]:

```
L.remove(9)
L
```

Out[17]:

```
[42, -4, 0, 7, 12, 2, 9, 7, 11]
```

In [18]:

```
L.index(7)
```

Out[18]:

```
3
```

In [19]:

```
L.count(7)
```

Out[19]:

```
2
```

Lists can be concatenated using `+` . This produces a new list.

In [20]:

```
[1,2,3] + [4,5,6]
```

Out[20]:

```
[1, 2, 3, 4, 5, 6]
```